

AegisAuth API By COHNDDS251F-001, 002, 003

<https://github.com/JNKODITHUWAKKU/AegisAuth-API/releases/tag/V1.1.0>

Contents

1	Product Overview	2
2	Core Features and Technical Requirements	2
2.1	Core Features	2
2.2	Technical Requirements	3
2.2.1	Software Dependencies	3
2.2.2	Infrastructure	3
2.2.3	Non-Functional Requirements	4
3	Data Strategy	4
3.1	Data Collection	4
3.2	Data Preprocessing and Bias Mitigation	4
3.3	Data Pipeline	5
4	Model Architecture, Training, and Optimization	5
4.1	Model Architecture	5
4.2	Training and Optimization	6
4.2.1	Hyperparameter Configuration	6
4.2.2	Training Progression	6
4.3	Performance Metrics	6
5	End-to-End Architecture and Deployment	8
5.1	End-to-End Architecture	8
5.2	Deployment Strategy	8
5.3	API Integration	9
6	Testing and Security	9
6.1	Model Testing	9
6.2	Product and Integration Testing	9
6.3	User Acceptance Testing (UAT)	10
6.4	Security Testing and Privacy	10
7	Conclusion	10

1 Product Overview

Traditional attendance and identification systems are plagued with issues of inefficiency. The majority of traditional attendance tracking methods are slow and prone to errors. Many of the first automated biometric identification systems suffered from a fundamental design issue. Most all of the original systems had to completely retrain the underlining classification models when introducing a new person to be identified. In addition, most of the original systems were extremely susceptible to spoofing attacks. Users could simply hold up photographs or digital screens in front of the cameras and bypass the systems.

AegisAuth API was created to address each of these issues. AegisAuth API is a real-time facial recognition and anti-spoofing deep-learning micro service. Unlike the rigid classification based models used in earlier systems which required constant retraining, AegisAuth uses facial embeddings. Facial embeddings allow for instant enrollment of new users. AegisAuth maps new students and employees into a mathematical vector space using an instant (1-shot) learning approach. As new students and employees join your organization, they are entered into the database in less than seconds without requiring changes to the core deep learning models.

Our primary customers will be educational institutions administering remote exams, corporate HR departments managing remote workers, and software developers creating secure remote access attendance applications.

The business value of AegisAuth API is substantial. Organizations can significantly reduce administrative costs and expenses related to manual monitoring and proctoring through automation of the verification process. Academic and Corporate Integrity will also be ensured as AegisAuth's custom Liveness Engine eliminates both Proxy Attendance and Spoofing. Additionally, since the product is delivered via a RESTful API, it is a very flexible Plug-and-Play module. Educational and corporate institutions do not need to abandon their existing infrastructure. Simply integrate AegisAuth's API with your existing Management Portal(s), and you immediately enhance your security capabilities.

2 Core Features and Technical Requirements

2.1 Core Features

AegisAuth has been developed with an entire array of Deep Learning capabilities specifically for Remote Authentication. The principal elements of functionality are grouped under the following core features:

- **Dynamic Live Identification:** Spoofing protection is the primary security concern in remote authentication. Unlike previous solutions which have relied upon detecting blinks or depth sensor technology, this solution utilizes a custom trained YOLOv8 object detection model. The model analyzes all contextual aspects of every video frame to identify if a detected face is indeed live or if it is simply a replication (for example, a photograph printed out and held up by the user or projected onto a screen).
- **One-Shot Enroll System:** This product employs FaceNet (InceptionResnet-V1), to generate 512-dimensional facial vector space representations. This enables users to enroll once in a single pass, eliminating the need to retrain the neural network. These vector representations can serve as unique mathematical identities.

- **Real-time Facial Location and Identification:** Each incoming frame from the client's webcam is rapidly evaluated utilizing YOLOv8n-face for the location of the bounding box surrounding the face. Once identified, the cropped facial region is evaluated via the embedding model and subsequently compared to the stored database entries via cosine similarity. The overall pipeline is rapid enough to provide live identification feedback to the final consumer.
- **Statelessness Restful API Integration:** AegisAuth was engineered as a modular microservices-based application utilizing FastAPI. AegisAuth has defined clear entry points for enrollment and verification. Developers may integrate the biometric engine into their existing web or mobile applications using standard HTTP calls.

2.2 Technical Requirements

Because of a need for balance between model performance and computation cost; the deep learning models chosen provide a perfect example of that. Although a large amount of compute power will always be provided by using a dedicated GPU (such as an NVIDIA GPU running CUDA), this was taken into account when designing the YOLOv8 nano model, which will still perform well even on most CPU's. The low overhead of this architecture provides the ability to deploy at the "edge" in applications such as stand-alone attendance kiosk systems where compute power may be limited.

2.2.1 Software Dependencies

Software required for the project include:

- **PyTorch:** The Deep Learning Framework used to support all tensor operation and serve models.
- **Ultralytics:** A framework that allows you to load and infer YOLOv8 models for Face Localization and Liveness Classification.
- **facenet-pytorch:** A framework that uses a pre-trained InceptionResnetV1 architecture to extract facial features.
- **FastAPI and Uvicorn:** An Asynchronous Web Server that supports multiple concurrent HTTP/HTTPS requests per second.
- **OpenCV and NumPy:** Used to decode image byte strings, convert color spaces, and manipulate arrays of tensors before sending them to the inference engine.

2.2.2 Infrastructure

The nature of our API is stateless. Therefore all identity parameters and threshold values are loaded from a centralized configuration file. User embeddings are written to disk as flat files. Because we have removed the requirement to store user state, we can now package our application into a Docker container and deploy to anywhere. Our application could therefore be hosted in a cloud provider such as AWS or GCP if we require scalability in a University Examination Portal context. However, it could also be hosted in an on-premises environment in the event we have strict data locality requirements due to our deployment being within a corporate environment.

2.2.3 Non-Functional Requirements

- **Performance and Latency:** Real-time video verification demands strict latency limits. The pipeline is optimized to process individual frames in under 200 milliseconds, ensuring the user interface does not lag or freeze during the authentication process.
- **Security and Privacy:** Raw image frames are never stored permanently by the system. Once an image is processed during enrollment, only the mathematical vector embedding is saved. This irreversible transformation protects user privacy and ensures compliance with modern data protection standards.
- **Scalability:** Because the deep learning inference does not rely on active session states, multiple instances of the API can sit behind a load balancer to handle thousands of concurrent exam-takers seamlessly.

3 Data Strategy

The integrity of any deep learning product depends fundamentally on the quality and handling of its underlying data. We implemented a rigorous data strategy encompassing collection, preprocessing, and automated pipelining to ensure our models learn effectively and operate without bias.

3.1 Data Collection

Our system relies on a hybrid data collection strategy. For the core facial feature extraction and face localization tasks, we leveraged models pre-trained on massive public datasets (such as VGGFace2 for the FaceNet architecture). However, anti-spoofing is highly contextual and depends on recognizing subtle environmental artifacts like screen glare, pixelation, or the lack of depth on a printed piece of paper.

To address this, we sourced a highly suitable, enterprise-grade dataset from Roboflow specifically annotated for face anti-spoofing detection. This dataset provides a robust collection of sample images categorized into two distinct classes: **real** (authentic human faces captured live) and **spoof** (faces presented to the camera via digital mobile screens or printed media). The conscious decision to leverage this specialized Roboflow dataset ensures that the liveness model learns the precise adversarial attack vectors it will face in real-world deployment, rather than relying on generalized data that might lack these specific spoofing artifacts. The dataset was programmatically fetched directly into our training pipeline utilizing the Roboflow Python API via our `get_dataset.py` script.

3.2 Data Preprocessing and Bias Mitigation

During the initial exploratory data analysis phase, we utilized our `check_class_imbalance.py` script and identified a significant disparity in the volume of samples between the ‘real’ and ‘spoof’ classes.

In deep learning classification tasks, feeding an imbalanced dataset into a neural network often results in a biased model that disproportionately predicts the majority class. In the context of an authentication system, a bias towards predicting faces as ‘real’ is a catastrophic security vulnerability.

To resolve this, we made an informed architectural decision to employ an undersampling technique via our `undersample.py` script. We intentionally pared down the majority class to perfectly match the sample size of the minority class. We chose undersampling over oversampling (such as SMOTE or simple duplication) because we wanted the model to learn exclusively from genuine, distinct variations in the data without introducing synthetic noise or overfitting to duplicated images. Furthermore, all images were strictly normalized and resized to a consistent resolution by the YOLO preprocessing pipeline to preserve aspect ratios and prevent spatial distortions during training.

3.3 Data Pipeline

To ensure repeatability and scalability, data ingestion was entirely automated. For the training phase, the `get_dataset.py` script programmatically fetches and structures the raw image data into the correct directory hierarchies required by the deep learning training framework.

For the production API, the data pipeline is optimized for real-time video processing. When a user interacts with the client application (as seen in `enrollment.py` or `live_verification.py`), OpenCV is utilized to capture continuous frames from the webcam. A critical engineering choice was made here to handle all frame extraction and JPEG encoding directly in system memory. By bypassing the physical hard drive and streaming memory buffers directly to the FastAPI endpoints via HTTP POST requests, we eliminate disk I/O bottlenecks. This pipeline design ensures the system meets the strict latency requirements necessary for a seamless, real-time user experience.

4 Model Architecture, Training, and Optimization

This section details the core deep learning components of the product, carrying the heaviest computational workload. We engineered a multi-model pipeline that handles face localization, facial embedding extraction, and spoof classification sequentially.

4.1 Model Architecture

To achieve a robust authentication system, a single monolithic model is insufficient. We constructed an ensemble architecture utilizing three distinct neural network models:

- **Face Localization (YOLOv8 Nano):** The first stage of the pipeline utilizes a highly compressed YOLOv8n-face model. Its sole responsibility is to quickly scan incoming frames and output precise bounding box coordinates for any detected faces. CNN-based object detectors like YOLO are exceptionally fast at localization compared to older methods like Haar Cascades.
- **Face Verification (InceptionResnetV1):** Once localized, the cropped face is passed into FaceNet, specifically the InceptionResnetV1 architecture pre-trained on the massive VGGFace2 dataset. Instead of classifying a face into a fixed number of identities (which would require retraining for every new user), this model acts as a feature extractor. It processes the input and outputs a 512-dimensional vector. Verification is then performed by computing the Cosine Similarity between this live vector and the stored anchor embeddings in the database.

- **Liveness Detection (YOLOv8 Small):** The most critical security feature is the anti-spoofing engine. We utilized transfer learning on a pre-trained YOLOv8s (Small) model. A deliberate architectural decision was made to use an object detection framework (YOLO) rather than a simple image classifier (like ResNet) for liveness. Image classifiers only look at the tightly cropped face, whereas YOLO evaluates the entire frame. This contextual awareness allows the model to detect spoofing artifacts in the background, such as the bezel of a mobile phone or the physical edges of a printed photograph held up to the camera.

4.2 Training and Optimization

The liveness detection model required custom training to adapt it from general object detection to specialized anti-spoofing. The training was conducted on cloud GPUs using the Ultralytics framework.

4.2.1 Hyperparameter Configuration

Based on the `args.yaml` configuration from our training runs, we optimized the model using the **AdamW** optimizer. AdamW was selected over standard SGD because it incorporates weight decay directly into the update step, which significantly helps in regularizing the network and preventing overfitting on our relatively constrained dataset. Key hyperparameters included:

- **Epochs:** 50
- **Batch Size:** 32
- **Image Resolution:** 640x640
- **Initial Learning Rate:** 0.001
- **Dropout:** 0.2 (Introduced to randomly zero out neurons, forcing the network to learn robust, redundant features rather than memorizing the training data).

4.2.2 Training Progression

An analysis of the training logs (`results.csv`) demonstrates a highly successful optimization phase. Over the 50 epochs, both the bounding box loss (`box_loss`) and the classification loss (`cls_loss`) exhibited a smooth, exponential decay, indicating that the AdamW optimizer efficiently navigated the gradient descent landscape without catastrophic oscillations.

4.3 Performance Metrics

To evaluate the model’s readiness for production, we relied on strict performance metrics aligned with the product’s core security functionality:

- **Mean Average Precision (mAP50):** The liveness model achieved an exceptional mAP50 of **0.973** by the final epoch. This metric confirms that the model correctly identifies and classifies **real** and **spoof** instances with high confidence when the Intersection over Union (IoU) threshold is 50%.

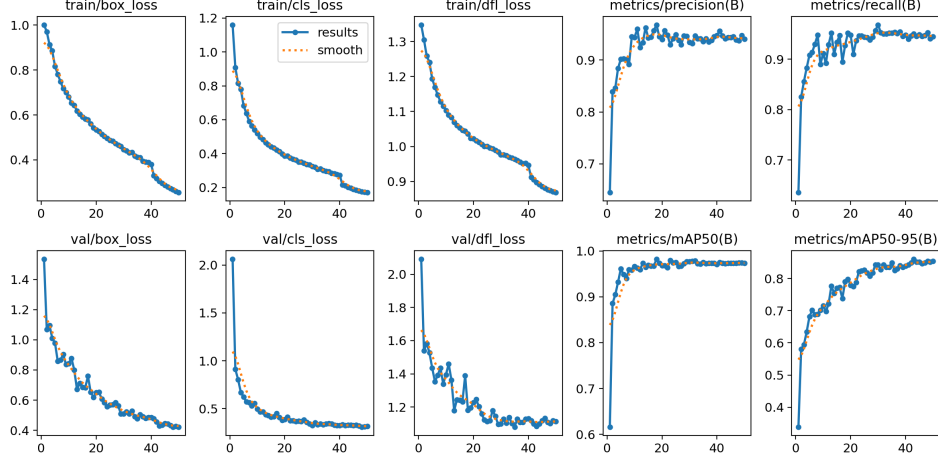


Figure 1: Training and Validation curves displaying Loss decay and mAP improvements over 50 epochs.

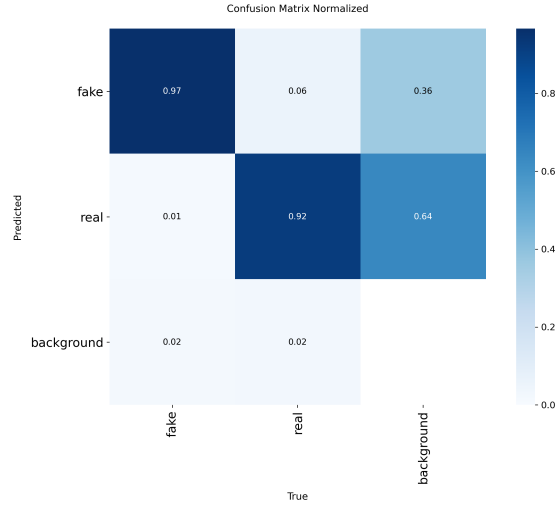


Figure 2: Normalized Confusion Matrix confirming high True Positive rates for both Real and Spoof classes.

- **Precision and Recall:** The model balanced out with a Precision of **0.940** and a Recall of **0.944**. In an authentication context, high precision is mandatory to minimize False Positives (incorrectly classifying a spoof as a real face).
- **Liveness Threshold (0.65):** During inference, we enforce a strict confidence threshold of 65%. If the YOLOv8s model's confidence for the **real** class drops below 0.65, the system immediately flags it as a spoof.
- **Match Tolerance (0.85):** For the facial embedding comparison, the Cosine Similarity must exceed 0.85. This tight tolerance ensures that two vectors are mathematically nearly identical before granting access, minimizing the risk of unauthorized entry.

5 End-to-End Architecture and Deployment

To bridge the gap between a standalone deep learning script and a production-ready product, the system was designed with a highly decoupled, microservice-based architecture.

5.1 End-to-End Architecture

The architecture separates the frontend user interfaces from the heavy deep learning backend. The client applications (such as the provided `enrollment.html` or Python OpenCV scripts) capture video frames and transmit them securely to the backend over HTTP. The FastAPI backend orchestrates the deep learning inference and communicates with a lightweight, file-based vector database.

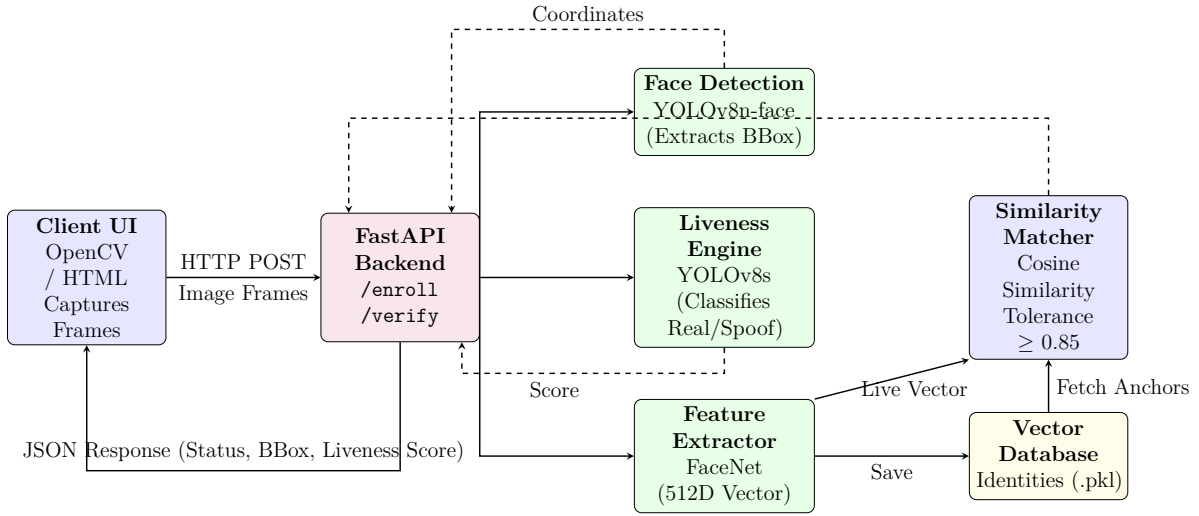


Figure 3: Detailed System Architecture illustrating the internal data flow from client frame capture, through the decoupled deep learning ensemble (Detection, Liveness, Extraction), to the vector similarity matching process.

The rationale behind this separation is modularity. By abstracting the deep learning models behind a REST API, we ensure that if the models need to be updated or swapped (e.g., upgrading from YOLOv8 to YOLOv9), the client-side code remains completely unaffected.

5.2 Deployment Strategy

Because the application is stateless and configuration-driven (managed via `config.py`), it supports highly flexible deployment strategies:

- **Cloud Deployment:** For large-scale integration into online examination portals (e.g., Moodle or Canvas), the FastAPI microservice can be containerized using Docker and deployed on cloud instances such as AWS EC2 or Google Cloud Platform (GCP). This allows the application to scale horizontally behind a load balancer during peak examination hours.
- **Edge Deployment:** Conversely, for localized physical deployment (e.g., a smart attendance kiosk at a university entrance), the system can be deployed directly onto

edge devices like the NVIDIA Jetson Nano. We deliberately chose the YOLOv8-nano model for face detection to ensure the inference runs smoothly on such resource-constrained IoT devices without the latency overhead of a round-trip to the cloud.

5.3 API Integration

The system serves its capabilities through two primary REST API endpoints, designed to be consumed by any modern web or mobile framework:

- **/enroll:** This endpoint accepts a multipart form containing multiple image frames and a `student_id`. It acts as the gatekeeper, first verifying liveness, and then saving the generated 512-dimensional facial embeddings securely into the database.
- **/verify:** This endpoint is called continuously by the client during authentication. It accepts a single frame, runs the detection-liveness-verification pipeline, and returns a structured JSON payload containing the `status` (Verified, Unregistered, or Spoof Detected), the `liveness_score`, and the bounding box coordinates for rendering UI overlays.

This plug-and-play API design means universities do not need to build biometric systems from scratch; they simply route their existing portal’s camera feeds to our `/verify` endpoint.

6 Testing and Security

To guarantee that the AegisAuth API performs reliably in unpredictable, real-world conditions, a comprehensive testing methodology was executed across all layers of the product stack.

6.1 Model Testing

The deep learning models were rigorously evaluated during the training phase using isolated validation datasets. As evidenced by the high mAP metrics, the YOLO liveness engine consistently generalizes to unseen data rather than memorizing the training set. Cross-validation was continuously monitored via loss curves to ensure that the network did not overfit, maintaining its performance integrity when exposed to novel faces and new physical environments.

6.2 Product and Integration Testing

Beyond isolated model benchmarks, the end-to-end software pipeline required extensive product testing. Unit tests and integration tests were conducted on the API utilizing the provided `enrollment.py` and `live_verification.py` client scripts. These tests confirmed that the FastAPI endpoints correctly decode inbound multipart image streams, pass tensors seamlessly between the PyTorch models, and return structured JSON responses within the 200-millisecond latency budget, ensuring continuous feedback without server timeout errors.

6.3 User Acceptance Testing (UAT)

For a biometric product, the user experience is equally as important as the algorithmic accuracy. User Acceptance Testing was conducted utilizing the HTML/JS web interfaces under varying environmental conditions such as low lighting, high glare, and different camera angles. The UI was designed to provide immediate, color-coded visual feedback (e.g., drawing a red bounding box for a spoof attempt and a green box for verified identities). UAT feedback confirmed that the system is intuitive; users instinctively adjust their position if the system flags poor lighting, minimizing administrative support tickets.

6.4 Security Testing and Privacy

Security is the foundational pillar of the AegisAuth API. Our testing heavily focused on adversarial robustness. The system was actively tested against common attack vectors, such as presenting high-resolution printed photographs and digital playback on mobile screens. The custom liveness engine successfully intercepted these spoof attempts by analyzing planar depths and screen glares.

Furthermore, data privacy was fundamentally engineered into the architecture. A primary advantage over legacy systems is that AegisAuth never stores raw facial imagery after the initial enrollment phase. Human faces are mathematically compressed into irreversible 512-dimensional vector arrays and saved as encrypted pickle files. Even in the highly unlikely event of a database breach, malicious actors would only recover abstract mathematical arrays, making it impossible to reverse-engineer or reconstruct the users' original facial images.

7 Conclusion

The AegisAuth API represents a significant leap forward for remote biometric authentication. By transitioning away from rigid, monolithic classification architectures and embracing a dynamic, one-shot embedding system paired with state-of-the-art liveness detection, we have solved the most pressing issues facing online examination and attendance portals today.

Our product is secure, inherently scalable, and designed with a privacy-first mindset. Through its lightweight API microservice architecture, it offers institutions a cost-effective, plug-and-play upgrade to their existing infrastructure. AegisAuth does not just automate authorization; it mathematically guarantees academic and corporate integrity in a remote world.

* * *